

CHOICE Forge — Layer 2 Explainer

What the CRF actually does: the model itself, BIO tagging, why it trains on `source_text` instead of `value`, multi-span decoding, and where confidence scores come from.

Source line from the architecture doc: [Layer 2] CRF, trained on this dataset (`train_crf.py -> role_tagger.joblib`) → per-token BIO tagging (B-ACTOR, I-ACTOR, B-INTENT, ...) trained against `source_text`, not `value`. Extractive job only. Handles multi-span roles natively.

LAYER 2 • CONCEPT 1

CRF (Conditional Random Field) (the model itself)

In plain terms

Picture reading a sentence word by word with a highlighter, deciding for each word “is this part of something important, and if so what kind” — but unlike a naive highlighter, it remembers what it just marked, so it doesn't randomly switch colors mid-phrase.

Technically

A CRF is a statistical sequence-labeling model. Given a sequence of per-token feature vectors (Layer 1's output — POS, dependency relation, lemma, entity type), it predicts a label for every token by modeling the probability of the **entire label sequence** given the features, including learned transition weights between adjacent labels — so I-ACTOR right after B-ACTOR is far more likely than I-ACTOR appearing out of nowhere after 0. Trained via L-BFGS optimization to maximize the likelihood of the 100 rows' gold label sequences.

```
import sklearn_crfsuite

crf = sklearn_crfsuite.CRF(
    algorithm="lbfgs",
    c1=0.1, c2=0.1,           # L1/L2 regularization -- guards against
                              # overfitting on only 100 rows
    max_iterations=150,
    all_possible_transitions=True,
)
crf.fit(X_train, y_train)    # X_train: per-token feature dicts
                             # y_train: per-token BIO tags
```

Why it matters in this pipeline

This is the object saved as `role_tagger.joblib`. It replaces the old hand-written if/else rules — instead of writing “if prep is 'for', it's scope,” it learns which feature combinations predict each role and generalizes to phrasing it wasn't explicitly given a rule for.

LAYER 2 • CONCEPT 2

BIO tagging (the label format)

In plain terms

A role like *scope* can span multiple words (“the SMB tier”), so you need a way to say not just “this word belongs to *scope*” but “this word starts the *scope* phrase” vs “this word continues it.” BIO is that system: B = Beginning, I = Inside (continues the same phrase), O = Outside (not part of any role).

Technically

BIO (also called IOB) is the standard way to reduce span-labeling to per-token classification. A run of one B-ROLE followed by zero or more I-ROLE decodes back into one span. `row_to_bio()` builds exactly these tags from the gold data:

```
def row_to_bio(doc, row):
    tags = ["O"] * len(doc)
    for role in ROLES:
        info = row["choice"][role]
        if info["status"] == "missing":
            continue
        for needle in as_span_list(info["source_text"]):
            for cs, ce in find_all_char_spans(row["input_query"], needle):
                span = doc.char_span(cs, ce, alignment_mode="expand")
                if span and all(tags[t.i] == "O" for t in span):
                    for i, tok in enumerate(span):
                        tags[tok.i] = f"{'B' if i == 0 else 'I'}-{role.upper()}"
    return tags

# "...for the SMB tier before..." with scope's source_text = "the SMB tier":
# the -> B-SCOPE   SMB -> I-SCOPE   tier -> I-SCOPE
```

Why it matters in this pipeline

This is what makes multi-word roles possible with a per-token classifier at all. Without BIO you could only ever say “this one word is SCOPE,” never “these three words together form one SCOPE phrase.”

Trained against source_text, not value (the extractive design decision)

In plain terms

The model is only ever taught to point at words already sitting in the sentence, never to write new words that aren't there. It's trained as a highlighter, not a writer.

Technically

value is frequently a paraphrase (89/100 rows for intent) while source_text is a verbatim substring 100% of the time, across all 655 checks in the dataset. row_to_bio() deliberately looks up info["source_text"], never info["value"] — training against value would mean most gold labels silently fail to align to any tokens at all (the span-finder just returns nothing for text that isn't literally present), actively corrupting the training set rather than just underperforming.

```
def find_all_char_spans(query, needle):
    spans = []
    q_low, n_low = query.lower(), needle.lower().strip()
    start = 0
    while True:
        idx = q_low.find(n_low, start)
        if idx == -1:
            break
        spans.append((idx, idx + len(n_low)))
        start = idx + len(n_low)
    return spans
```

Why it matters in this pipeline

This is the single decision that makes the whole three-layer split work. It's why Layer 2 gets to be “just” a CRF instead of also having to understand paraphrase — that job is pushed entirely to Layer 3 (T5).

Multi-span roles / decoding

In plain terms

Sometimes one field shows up in two separate places in the same sentence — like “25,000 customers” and “under 350” both being part of magnitude. BIO doesn't need anything special for this: it marks two separate B/I runs with the same role name, and decoding just collects all of them instead of assuming there's only ever one.

Technically

decode_bio_multi() walks the predicted tag sequence and, for every B-ROLE, greedily consumes the following I-ROLE run as one span, then keeps scanning — a second B-ROLE later in the sequence becomes an independent second span, not a merge or an overwrite.

```
def decode_bio_multi(doc, tags, marginals):
    spans = {role: [] for role in ROLES}
```

```
i = 0
while i < len(tags):
    tag = tags[i]
    if tag.startswith("B-"):
        role = tag[2:].lower()
        start = i
        j = i + 1
        while j < len(tags) and tags[j] == f"I-{{tag[2:]}}":
            j += 1
        spans[role].append(doc[start:j].text)
        i = j
    else:
        i += 1
return spans
```

Why it matters in this pipeline

choice_015 needed exactly this — magnitude grounded in both “25,000” and “under ■350” in the same query — and the pipeline captures both instead of only the first, or mangling the text in between into one nonsensical span.

Confidence via marginal probability

In plain terms

Alongside its best guess for each word's label, the model can also say how sure it is — not just “I think this is SCOPE” but “I think this is SCOPE, 92% confident.” That number becomes the confidence you see in the output table.

Technically

`crf.predict_single()` gives the single best label sequence (Viterbi decoding — maximizes overall sequence likelihood). `crf.predict_marginals_single()` instead computes, per token and per possible label, that label's probability independent of which exact full sequence wins (via the forward-backward algorithm). `pipeline.py` averages the marginal probability of the predicted tag across all tokens in a span to get one confidence number per role.

```
tags = crf.predict_single(feats)
# ['O', 'O', 'B-SCOPE', 'I-SCOPE', 'I-SCOPE', 'O', ...]

marginals = crf.predict_marginals_single(feats)
# marginals[2] = {'O': 0.02, 'B-SCOPE': 0.91, 'B-TIME': 0.04, ...}
```

Why it matters in this pipeline

This is where the extraction-half confidence number in your final table actually comes from — it reflects how sure the trained model is on that specific query, not a fixed guess.

Quick reference

Concept	Where it lives	Input	Output
CRF model	<code>role_tagger.joblib (train_crf.py)</code>	per-token feature dicts	best BIO tag sequence
BIO tagging	<code>row_to_bio()</code>	gold <code>source_text</code> spans	B-ROLE / I-ROLE / O per token
<code>source_text</code> training target	<code>row_to_bio()</code> / <code>find_all_char_spans()</code>	<code>input_query</code> + <code>source_text</code>	aligned char spans
Multi-span decoding	<code>decode_bio_multi()</code> (<code>pipeline.py</code>)	predicted BIO tags	list of spans per role
Marginal confidence	<code>predict_marginals_single()</code>	per-token feature dicts	per-tag probability at each position